

I. Résumé d'exploration

Dans cette partie, nous allons nous intéresser à l'exploration des données.

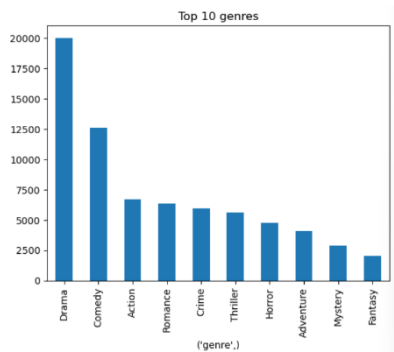


Fig1

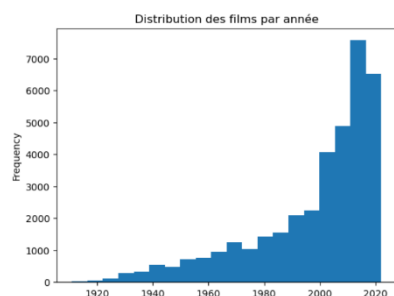


Fig 2

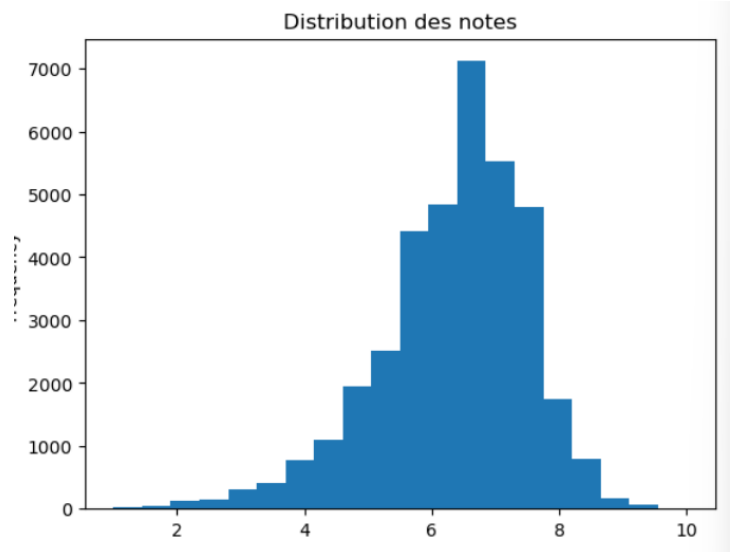
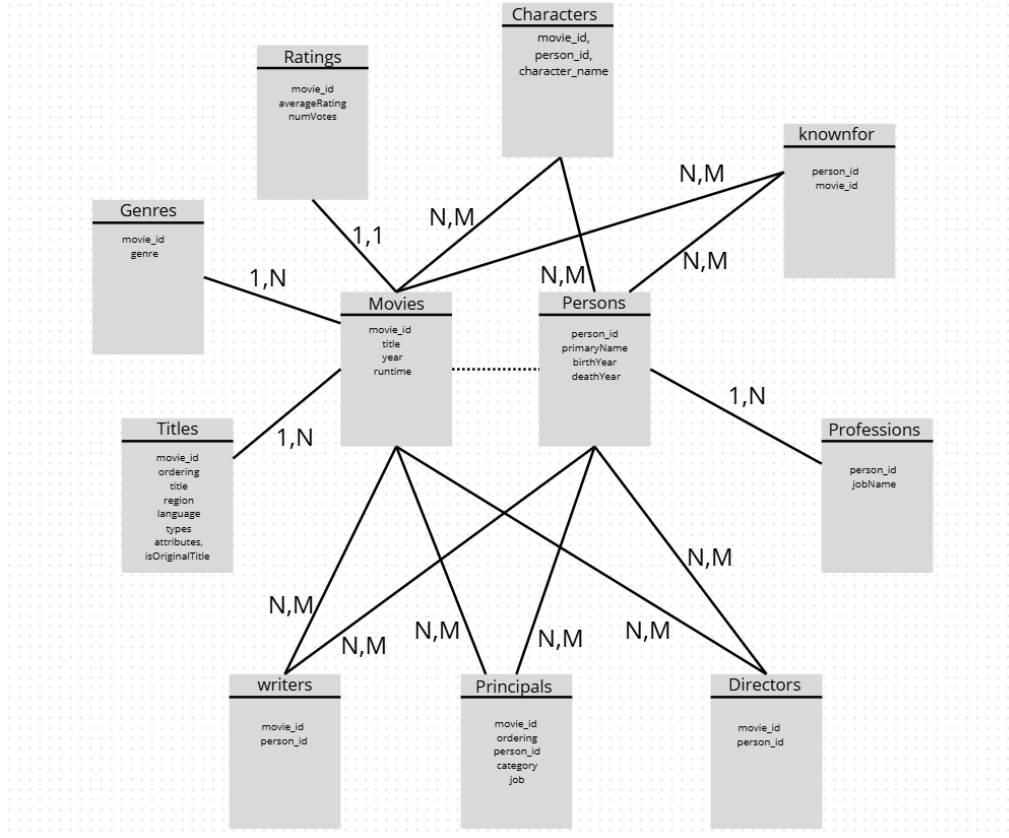


Fig 3

On peut grâce à la fig1, établir le top 10 des genres les plus fréquents. Le genres le plus fréquent est le Drame suivi de la Comédie. Ensuite, on peut remarquer grâce à la fig 2 que le nombre de films produits augmente grandement à partir des années 2000. La figure 3 nous permet quant à elle d'établir que la majorité des films produits est évalué à 7/10 et que la répartition des notes peut être assimilée à une courbe gaussienne.

II. Diagramme ER du schéma relationnel



III. Explications des requêtes SQL

Dans cette partie, nous allons expliquer les requêtes implémentées dans queries.py :

Requête Q1: Filmographie d'un acteur

Cette requête récupère l'ensemble des films dans lesquels un acteur donné a joué. Pour cela, elle joint cinq tables : movies pour les informations du film, principals pour relier les films aux personnes, persons pour filtrer sur le nom de l'acteur, characters pour récupérer les rôles joués, et ratings pour ajouter la note moyenne. L'utilisation d'un LEFT JOIN avec characters permet d'inclure même les films où l'information du personnage n'est pas disponible. Le filtrage par LIKE permet de rechercher un nom partiel.

Requête Q2 : Top N films d'un genre sur une période

Cette requête sélectionne les films appartenant à un genre donné et sortis entre deux années. Elle joint movies, genres et ratings. Les films sont triés par note moyenne (averageRating) dans l'ordre décroissant, puis la clause LIMIT permet de ne conserver

que les N meilleurs. Cette requête illustre un usage typique de filtres + tri sur une métrique.

Requête Q3 : Acteurs ayant joué plusieurs rôles dans un même film

Cette requête examine la table `characters`, qui peut contenir plusieurs entrées par acteur et par film. En regroupant (`GROUP BY`) par (`movie_id`, `person_id`) et en comptant le nombre de rôles, elle identifie les personnes ayant joué plus d'un personnage dans un seul film. Le `HAVING` impose que le compteur soit supérieur à 1. Cela met en évidence les acteurs multi-rôles.

Requête Q4 : Collaborations entre un acteur et des réalisateurs

Cette requête cherche avec quels réalisateurs un acteur donné a travaillé et combien de fois. Elle joint les tables `principals`, `persons` (pour identifier l'acteur), `directors` (pour obtenir la liste des réalisateurs du film), et à nouveau `persons` pour récupérer leur nom. L'utilisation d'un `GROUP BY` sur le réalisateur permet de compter le nombre de collaborations. On identifie ainsi les réalisateurs les plus fréquents pour un acteur.

Requête Q5 : Genres populaires

Cette requête calcule la popularité des genres en regroupant par genre. Elle fait une moyenne des notes (`AVG`) et un comptage des films (`COUNT`). Le `HAVING` permet de sélectionner uniquement les genres suffisamment représentés (plus de 50 films) et bien notés (moyenne > 7). C'est une analyse classique d'agrégation.

Requête Q6 : Évolution de carrière d'un acteur par décennie

On utilise ici une CTE (`WITH films AS (...)`) qui sélectionne tous les films d'un acteur et leurs notes. On calcule ensuite la décennie en utilisant $(year/10)*10$, puis on groupe par décennie pour compter les films et calculer la moyenne des notes pour chaque période. C'est une analyse temporelle sur la carrière d'un acteur.

Requête Q7 : Classement des meilleurs films par genre

Cette requête utilise une *window function* (`ROW_NUMBER() OVER (PARTITION BY genre ORDER BY rating DESC)`) pour attribuer un rang à chaque film dans chaque genre selon sa note. Ensuite, on filtre pour garder les 3 premiers films par genre.

Requête Q8 : Personnes ayant eu une carrière propulsée

Cette requête cherche les personnes ayant joué dans des films peu populaires avant (moins de 200k votes) et dans des films très populaires après (plus de 200k votes). Les expressions `SUM(condition)` comptent les cas où la condition est vraie afin de simuler des compteurs conditionnels. Le `HAVING` garantit que les deux situations existent pour la même personne.

Requête Q9 : Artistes "multi-skill" (acteur + réalisateur)

Cette requête identifie les personnes qui apparaissent à la fois comme acteurs (dans `principals`) et comme réalisateurs (dans `directors`). En joignant `persons`, `principals` et

directors, puis en regroupant par personne, on compte le nombre de films où la personne occupe les deux rôles. Le critère `HAVING nb_films >= 3` permet d'isoler les profils les plus polyvalents.

IV. Table de benchmark

Nous avons ensuite le tableau de benchmark :

Requête	nom	Sans index (ms)	Avec Index (ms)	Gain (%)
Q1	Filmography	1.41	0.35	74.9
Q2	Top N genre	0.32	0.20	36.9
Q3	Acteurs multi-rôles	0.31	0.16	47.6
Q4	Collaborations	0.39	0.41	-3.1
Q5	Genres populaires	0.29	0.15	47.0
Q6	Évolution de carrière	0.44	0.36	17.3
Q7	Classement par genre	0.45	0.20	56.2
Q8	Carrière propulsée	0.34	0.15	54.9
Q9	Requête libre	0.29	0.15	49.0

On constate que le temps sans index sont entre 0.29 ms et 1.41 ms, ce qui semble cohérent car notre ensemble de données est relativement petit. On remarque également que les valeurs de gains sont élevées, ce qui est un résultat attendu, même si une baisse de -3.1% est observée, justifiable par le fait que Q4 n'utilise pas d'index.

V. Index créés et justification

Dans cette partie, nous nous intéresserons à la création des index et expliquerons pourquoi ils sont utiles :

1. `idx_principals_movie` (`principals.movie_id`)

Cet index est très utile pour les jointures fréquentes entre movies et principals, notamment dans les requêtes Q1, Q4, Q6 et Q9. Comme principals est l'une des plus grosses tables, cet index réduit fortement le coût d'accès aux lignes associées à un film.

2. `idx_principals_person` (`principals.person_id`)

Cet index accélère la recherche des films d'un acteur ou d'une personne, ce qui impacte directement Q1, Q4, Q6, Q8 et Q9. Sans index, SQLite devrait parcourir la table entière pour chaque nom.

3. idx_genres_movie (genres.movie_id)

Cet index est indispensable pour Q2 et Q7, qui filtrent ou classent les films par genre. Comme genres contient plusieurs genres par film, un accès direct sur movie_id accélère toutes les jointures.

4. idx_titles_movie (titles.movie_id)

Bien que non utilisé dans les requêtes Q1–Q9, cet index est essentiel pour les futures opérations du projet (notamment Phase 4 dans Django), car les titres alternatifs sont souvent consultés via movie_id.

5. idx_ratings_rating (ratings.averageRating)

Optimise le tri des films par note dans Q2, Q5 et Q7. Toute requête utilisant ORDER BY averageRating en tire profit.

6. idx_movies_year (movies.year)

Accélère les filtres temporels dans Q2 et Q6, ainsi que les statistiques dans la suite du projet. Les recherches par période sont beaucoup plus efficaces.

7. idx_persons_name (persons.primaryName)

Essentiel pour toutes les recherches d'acteurs ou réalisateurs (Q1, Q4, Q6, Q8). Les requêtes LIKE '%nom%' sont coûteuses, mais un index sur la colonne permet quand même d'utiliser un préfiltrage.

8. idx_characters_movie et idx_characters_person

Ces deux index soutiennent particulièrement Q1 et Q3 notamment pour la recherche des rôles d'un film (movie_id) et la recherche des rôles d'une personne (person_id). Comme la table characters peut contenir beaucoup d'entrées, cet index rend les regroupements plus rapides.

9. idx_knownfor_person (known_for.person_id)

Bien qu'utilisé surtout dans les extensions du projet, il permet des recherches plus rapides sur les films emblématiques d'un acteur. Il contribue à l'optimisation globale de la base.